1. ∈ Greedy Numerical

1. Given Problem

We have a **4-armed bandit**:

- Actions: $1, 2, 3, 4$
- Initial action-value estimates:

$$Q_1(1) = Q_1(2) = Q_1(3) = Q_1(4) = 0$$

- Initial counts:

$$N_1(1) = N_1(2) = N_1(3) = N_1(4) = 0$$

The observed sequence is:

Time	Action A_t	Reward R_t
1	1	-1
2	1	+1
3	2	-2
4	2	+2
5	3	0

We have to determine:

1. **At which time steps did a random action definitely occur?**
2. **At which time steps could a random action have occurred?**

2. What is ε-greedy?

In ε-greedy action selection:

- With probability $1 - \epsilon$ → choose a **greedy action**
- With probability ϵ → choose an **action randomly**

For example, if:

$$\epsilon = 0.1$$

then:

$$P(\text{greedy}) = 0.9$$

and

$$P(\text{random}) = 0.1$$

Important point

If several actions have the same maximum Q-value, **all of them are greedy actions**.

Therefore, if the selected action is tied for the highest Q-value, we **cannot say for certain** that it was selected randomly.

It **could** have been selected randomly.

3. Sample-average formula

For a bandit problem, we update the estimated value using:

$$Q(A_t) = \frac{\text{sum of rewards received from } A_t}{\text{number of times } A_t \text{ was selected}}$$

An incremental form is:

$$Q_{new}(A_t) = Q_{old}(A_t) + \frac{1}{N(A_t)}[R_t - Q_{old}(A_t)]$$

where:

- Q_{old} = previous estimated value
- R_t = current reward
- $N(A_t)$ = number of times action has been selected
- Q_{new} = updated estimated value

---

4. Time Step 1

Initially:

Action	Q-value
Action 1	0
Action 2	0
Action 3	0
Action 4	0

The given action is:

$$A_1 = 1$$

and reward:

$$R_1 = -1$$

Which actions are greedy?

The maximum Q-value is:

$$\max(0, 0, 0, 0) = 0$$

Therefore:

$$\{1, 2, 3, 4\}$$

are all greedy.

Action 1 is one of the greedy actions.

Therefore, action 1 **does not have to be random**.

It could have been selected because:

- ε-greedy exploitation selected Action 1, OR
- ε-greedy exploration randomly selected Action 1.

Update

Action 1 was selected once:

$$N(1) = 1$$

Therefore:

$$Q(1) = \frac{-1}{1}$$

$$\boxed{Q(1) = -1}$$

Current values:

Action	Q-value
1	-1
2	0
3	0
4	0

---

5. Time Step 2

Given:

$$A_2 = 1$$

Before selecting Action 1:

Action	Q-value
1	-1
2	0
3	0
4	0

The maximum value is:

$$\max(-1, 0, 0, 0) = 0$$

Therefore the greedy actions are:

$$\{2, 3, 4\}$$

But the selected action is:

$$A_2 = 1$$

Action 1 has:

$$Q(1) = -1$$

Therefore Action 1 is **not greedy**.

Hence:

Action 1 could not have been selected through exploitation.

Therefore it **must have been selected randomly**.

$$\boxed{\text{Time step } 2 = \text{DEFINITELY RANDOM}}$$

Update Action 1

Action 1 has now been selected twice.

$$N(1) = 2$$

Rewards received by Action 1:

$$-1,\ +1$$

Therefore:

$$Q(1) = \frac{-1+1}{2}$$

$$\boxed{Q(1) = 0}$$

Now:

Action	Q-value
1	0
2	0
3	0
4	0

6. Time Step 3

Given:

$$A_3 = 2$$

Before selecting Action 2:

Action	Q-value
1	0
2	0
3	0
4	0

The maximum is:

$$\max(0, 0, 0, 0) = 0$$

All four actions are greedy.

$$\{1, 2, 3, 4\}$$

Action 2 is therefore a greedy action.

So Action 2 **could have been selected through exploitation**.

But it could also have been selected through exploration.

Therefore:

$$\boxed{\text{Time step } 3 = \text{COULD BE RANDOM}}$$

It is **not definitely random**.

Update

Reward:

$$R_3 = -2$$

Action 2 has been selected once:

$$N(2) = 1$$

Therefore:

$$Q(2) = \frac{-2}{1}$$

$$\boxed{Q(2) = -2}$$

Current values:

Action	Q-value
1	0
2	-2
3	0
4	0

7. Time Step 4

Given:

$$A_4 = 2$$

Before selecting Action 2:

Action	Q-value
1	0
2	-2
3	0
4	0

Maximum:

$$\max(0, -2, 0, 0) = 0$$

Greedy actions:

$$\{1, 3, 4\}$$

But selected action:

$$A_4 = 2$$

and:

$$Q(2) = -2$$

Therefore Action 2 is **not greedy**.

So it could not have been selected through exploitation.

Hence:

$$\boxed{\text{Time step } 4 = \text{DEFINITELY RANDOM}}$$

Update

Action 2 has now been selected twice.

Rewards:

$$-2,\ +2$$

Therefore:

$$Q(2) = \frac{-2+2}{2}$$

$$\boxed{Q(2) = 0}$$

Current values:

Action	Q-value
1	0
2	0
3	0
4	0

---

8. Time Step 5

Given:

$$A_5 = 3$$

Before selecting Action 3:

Action	Q-value
1	0
2	0
3	0
4	0

Again, all actions have the same maximum value:

$$\max(0, 0, 0, 0) = 0$$

Therefore:

$$\{1, 2, 3, 4\}$$

are all greedy.

Action 3 is one of the greedy actions.

Therefore Action 3 **could have been selected through exploitation**.

But it could also have been selected randomly.

Hence:

$$\boxed{\text{Time step } 5 = \text{COULD BE RANDOM}}$$

Update

Reward:

$$R_5 = 0$$

Action 3 was selected once:

$$N(3) = 1$$

Therefore:

$$Q(3) = \frac{0}{1} = 0$$

Final values:

Action	Number of selections	Rewards	Final Q
1	2	-1, +1	0
2	2	-2, +2	0
3	1	0	0
4	0	—	0

9. Complete Numerical Table

This is the most important table for examination.

Time	Action	Reward	Q-values before action	Greedy actions	Random?
1	1	-1	(0, 0, 0, 0)	{1,2,3,4}	**Could be**
2	1	+1	(-1, 0, 0, 0)	{2,3,4}	**Definitely**
3	2	-2	(0, 0, 0, 0)	{1,2,3,4}	**Could be**
4	2	+2	(0, -2, 0, 0)	{1,3,4}	**Definitely**
5	3	0	(0, 0, 0, 0)	{1,2,3,4}	**Could be**

---

10. Final Answer

Definitely random

$$\boxed{t = 2,\ 4}$$

Could have been random

$$\boxed{t = 1,\ 3,\ 5}$$

Therefore:

> **The actions at time steps 2 and 4 were definitely selected randomly. The actions at time steps 1, 3 and 5 could have been selected randomly.**

11. Why is Time Step 1 only "Could Be Random"?

This is a very important concept.

At $t = 1$:

$$Q(1) = Q(2) = Q(3) = Q(4) = 0$$

Suppose:

$$\epsilon = 0.1$$

There are two possibilities:

Exploitation

The algorithm chooses a greedy action.

Since **all four actions are tied**, Action 1 can be chosen.

Exploration

The algorithm randomly chooses an action.

It can also randomly choose Action 1.

Therefore we cannot determine which happened.

So:

$$\boxed{t = 1 \text{ could be random, but is not definitely random}}$$

Exactly the same logic applies to $t = 3$ and $t = 5$.

12. Python Implementation

```
# Epsilon-Greedy Multi-Armed Bandit
# Given sequence:
# A = [1, 1, 2, 2, 3]
# R = [-1, 1, -2, 2, 0]

# Number of actions
k = 4
```

```
# Initial action-value estimates
Q = [0.0] * k

# Number of times each action is selected
N = [0] * k

# Given actions
actions = [1, 1, 2, 2, 3]

# Given rewards
rewards = [-1, 1, -2, 2, 0]

for t in range(len(actions)):

    # Convert action number to Python index
    action = actions[t] - 1

    # Current reward
    reward = rewards[t]

    # Find maximum Q-value
    max_Q = max(Q)

    # Find all greedy actions
    greedy_actions = [
        i + 1 for i in range(k)
        if Q[i] == max_Q
    ]

    # Check whether selected action is greedy
```

```
    if actions[t] in greedy_actions:
        random_status = "Could be random"
    else:
        random_status = "Definitely random"

    # Update number of times action was selected
    N[action] += 1

    # Sample-average update
    Q[action] = Q[action] + (
        1 / N[action]
    ) * (reward - Q[action])

    # Display result
    print("Time:", t + 1)
    print("Action:", actions[t])
    print("Reward:", reward)
    print("Greedy actions:", greedy_actions)
    print("Status:", random_status)
    print("Updated Q-values:", Q)
    print("-" * 50)
```

18. Expected Output

```
Time: 1
Action: 1
Reward: -1
Greedy actions: [1, 2, 3, 4]
Status: Could be random
Updated Q-values: [-1.0, 0.0, 0.0, 0.0]
```

Time: 2

Action: 1

Reward: 1

Greedy actions: [2, 3, 4]

Status: Definitely random

Updated Q-values: [0.0, 0.0, 0.0, 0.0]

Time: 3

Action: 2

Reward: -2

Greedy actions: [1, 2, 3, 4]

Status: Could be random

Updated Q-values: [0.0, -2.0, 0.0, 0.0]

Time: 4

Action: 2

Reward: 2

Greedy actions: [1, 3, 4]

Status: Definitely random

Updated Q-values: [0.0, 0.0, 0.0, 0.0]

Time: 5

Action: 3

Reward: 0

Greedy actions: [1, 2, 3, 4]

Status: Could be random

Updated Q-values: [0.0, 0.0, 0.0, 0.0]

2. UCB Numerical

1. Given Problem

After a total of **1000 steps**, we have:

Machine	Q-value $Q(M_i)$	Number of times played $N_a(M_i)$
M_1	1	250
M_2	1	350
M_3	1	400

Total number of steps:

$$N = 1000$$

The UCB formula is:

$$\boxed{UCB_i = Q(M_i) + c\sqrt{\frac{\ln N}{N_a(M_i)}}}$$

where:

- $Q(M_i)$ = estimated reward/value of machine i
- N = total number of steps
- $N_a(M_i)$ = number of times machine i has been selected
- c = exploration parameter
- $\ln$ = natural logarithm

From your handwritten solution, we use:

$$\boxed{c = 2}$$

2. Why do we use UCB?

UCB balances two things:

Exploitation

Choose the machine that currently has a high Q-value.

Exploration

Give some additional bonus to machines that have been played fewer times.

The formula is:

$$\underbrace{Q(M_i)}_{\text{Exploitation}} + \underbrace{c\sqrt{\frac{\ln N}{N_a(M_i)}}}_{\text{Exploration}}$$

Therefore:

$$UCB = \text{Exploitation value} + \text{Exploration bonus}$$

---

3. Step 1 — Calculate $\ln(N)$

We have:

$$N = 1000$$

Therefore:

$$\ln(1000) = 6.907755$$

So our formula becomes:

$$UCB_i = Q(M_i) + 2\sqrt{\frac{6.907755}{N_a(M_i)}}$$

---

4. Calculate UCB for Machine M_1

Given:

$$Q(M_1) = 1$$

and

$$N_a(M_1) = 250$$

Using:

$$UCB_1 = Q(M_1) + 2\sqrt{\frac{\ln(1000)}{250}}$$

Substitute the values:

$$UCB_1 = 1 + 2\sqrt{\frac{6.907755}{250}}$$

First calculate:

$$\frac{6.907755}{250} = 0.027631$$

Now take square root:

$$\sqrt{0.027631} = 0.166226$$

Multiply by $c = 2$:

$$2(0.166226) = 0.332452$$

Finally:

$$UCB_1 = 1 + 0.332452$$

$$\boxed{UCB_1 = 1.33245}$$

Approximately:

$$\boxed{UCB_1 \approx 1.33}$$

---

5. Calculate UCB for Machine M_2

Given:

$$Q(M_2) = 1$$

and

$$N_a(M_2) = 350$$

Formula:

$$UCB_2 = 1 + 2\sqrt{\frac{\ln(1000)}{350}}$$

Substitute:

$$UCB_2 = 1 + 2\sqrt{\frac{6.907755}{350}}$$

Calculate:

$$\frac{6.907755}{350} = 0.0197364$$

Square root:

$$\sqrt{0.0197364} = 0.140486$$

Multiply by 2:

$$2(0.140486) = 0.280972$$

Therefore:

$$UCB_2 = 1 + 0.280972$$

$$\boxed{UCB_2 = 1.28097}$$

Approximately:

$$\boxed{UCB_2 \approx 1.28}$$

This is the **1.28** shown in your handwritten solution.

---

6. Calculate UCB for Machine M_3

Given:

$$Q(M_3) = 1$$

and:

$$N_a(M_3) = 400$$

Formula:

$$UCB_3 = 1 + 2\sqrt{\frac{\ln(1000)}{400}}$$

Substitute:

$$UCB_3 = 1 + 2\sqrt{\frac{6.907755}{400}}$$

Calculate:

$$\frac{6.907755}{400} = 0.0172694$$

Square root:

$$\sqrt{0.0172694} = 0.131413$$

Multiply by 2:

$$2(0.131413) = 0.262826$$

Therefore:

$$UCB_3 = 1 + 0.262826$$

$$\boxed{UCB_3 = 1.26283}$$

Approximately:

$$\boxed{UCB_3 \approx 1.26}$$

This matches the **1.26** shown in your handwritten solution.

7. Final UCB Values

We now have:

Machine	Q-value	Times played	Exploration Bonus	UCB
M_1	1.00	250	0.33245	**1.33245**
M_2	1.00	350	0.28097	**1.28097**
M_3	1.00	400	0.26283	**1.26283**

Therefore:

$$UCB_1 = 1.33245$$

$$UCB_2 = 1.28097$$

$$UCB_3 = 1.26283$$

---

8. Which Machine Will UCB Select?

UCB selects the machine having the **maximum UCB value**.

Therefore:

$$M^* = \arg\max(UCB_1, UCB_2, UCB_3)$$

$$M^* = \arg\max(1.33245, 1.28097, 1.26283)$$

Clearly:

$$1.33245 > 1.28097 > 1.26283$$

Therefore:

$$\boxed{M_1 \text{ will be selected}}$$

Final Answer

$$\boxed{\text{Selected Machine} = M_1}$$

---

9. Why did M_1 get selected?

This is an important concept for understanding UCB.

All three machines have the same Q-value:

$$Q(M_1) = Q(M_2) = Q(M_3) = 1$$

So exploitation is identical.

The difference comes from the **exploration term**:

$$c\sqrt{\frac{\ln N}{N_a}}$$

Look at the number of times each machine has been played:

$$N_a(M_1) = 250$$

$$N_a(M_2) = 350$$

$$N_a(M_3) = 400$$

Machine M_1 has been played the **least number of times**.

Therefore, it gets the **largest exploration bonus**.

Hence:

$$UCB_1 > UCB_2 > UCB_3$$

and M_1 is selected.

---

10. Understanding the handwritten solution

Your handwritten image shows approximately:

$$Q_1 = 1 + 2\sqrt{\frac{\ln(1000)}{250}}$$

which gives:

$$Q_1 = 1.33$$

Similarly:

$$Q_2 = 1 + 2\sqrt{\frac{\ln(1000)}{350}}$$

gives:

$$Q_2 = 1.28$$

And:

$$Q_3 = 1 + 2\sqrt{\frac{\ln(1000)}{400}}$$

gives:

$$Q_3 = 1.26$$

Then:

$$A_t = \arg\max(1.33, 1.28, 1.26)$$

Therefore:

$$\boxed{A_t = M_1}$$

So the handwritten solution is essentially correct; the values **1.33, 1.28 and 1.26 are the calculated UCB scores**, rather than the original Q-values.

11. Python Implementation

```
import math

# Total number of steps
N = 1000

# Exploration parameter
c = 2

# Q-values of the three machines
Q = {
   "M1": 1,
   "M2": 1,
   "M3": 1
}

# Number of times each machine was played
N_a = {
   "M1": 250,
   "M2": 350,
   "M3": 400
}

# Calculate UCB for every machine
UCB = {}

for machine in Q:
   ucb = Q[machine] + c * math.sqrt(math.log(N) / N_a[machine])
   UCB[machine] = ucb
```

```
# Display UCB values
for machine in UCB:
    print(machine, "=", round(UCB[machine], 4))

# Select machine with maximum UCB
selected_machine = max(UCB, key=UCB.get)

print("\nSelected Machine:", selected_machine)
```

12. Expected Output

```
M1 = 1.3325
M2 = 1.2810
M3 = 1.2628

Selected Machine: M1
```